Constraint Programming

Alessandro Dal Palù

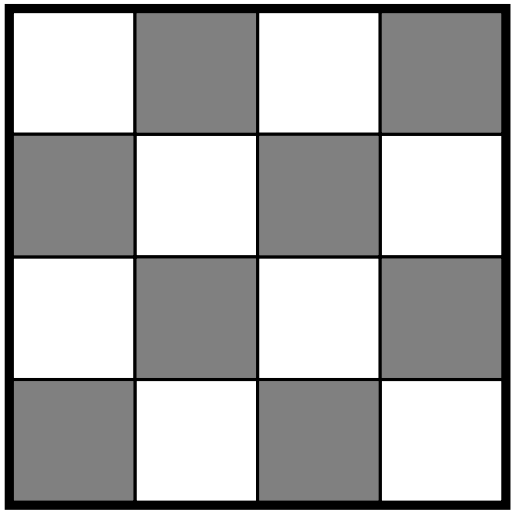
Università di Parma
Master's Degree in Computer Science

Parma 2025

Example

4-queens

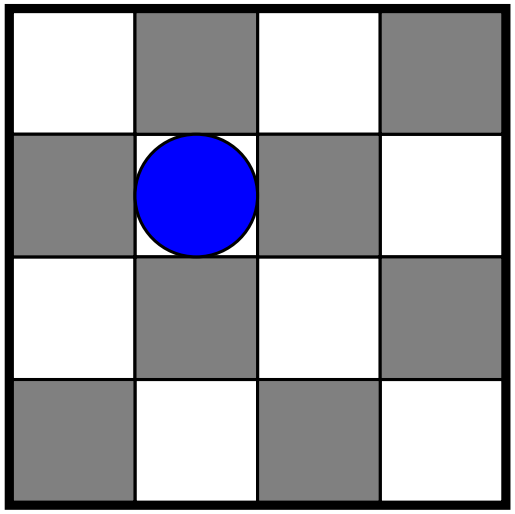
4-Queens: put 4 queens on a chessboard 4×4 in such a way as they do not attack each other



Example

4-queens

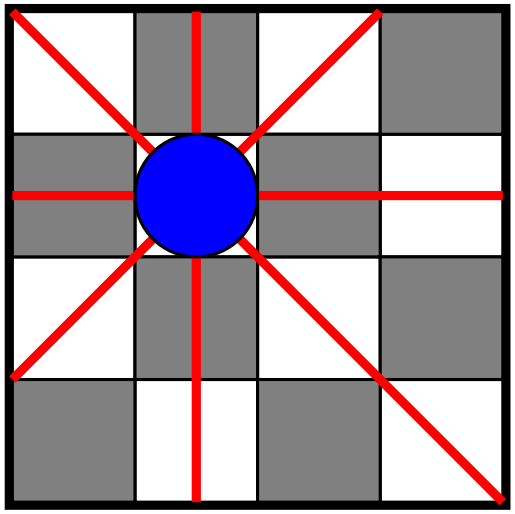
4-Queens: put 4 queens on a chessboard 4×4 in such a way as they do not attack each other



Example

4-queens

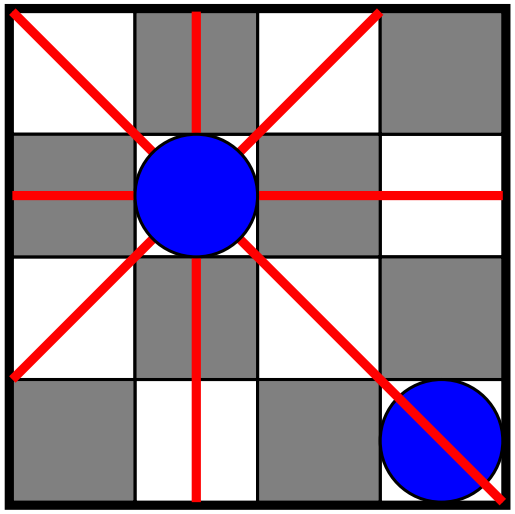
4-Queens: put 4 queens on a chessboard 4×4 in such a way as they do not attack each other



Example

4-queens

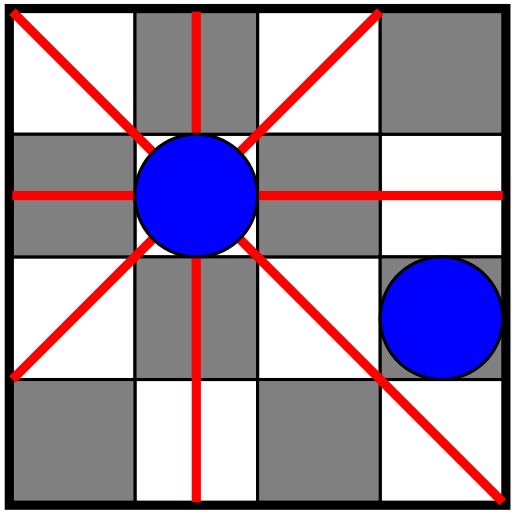
4-Queens: put 4 queens on a chessboard 4×4 in such a way as they do not attack each other



Example

4-queens

4-Queens: put 4 queens on a chessboard 4×4 in such a way as they do not attack each other

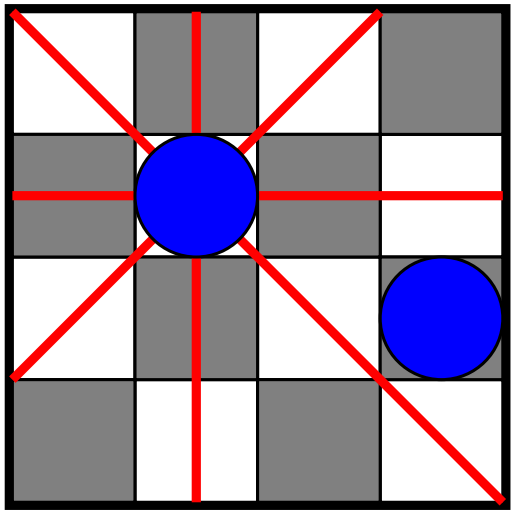


Example

4-queens

4-Queens: put 4 queens on a chessboard 4×4 in such a way that they do not attack one other

Let's start modeling the problem: Constraints?



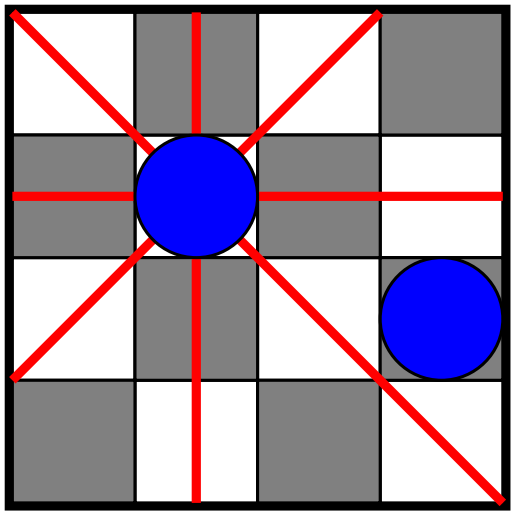
Example

4-queens

4-Queens: put 4 queens on a chessboard 4×4 in such a way that they do not attack one other

Let's start modeling the problem: Constraints?

Red lines characterize positions to be excluded from solutions (they depend on queen's positions)

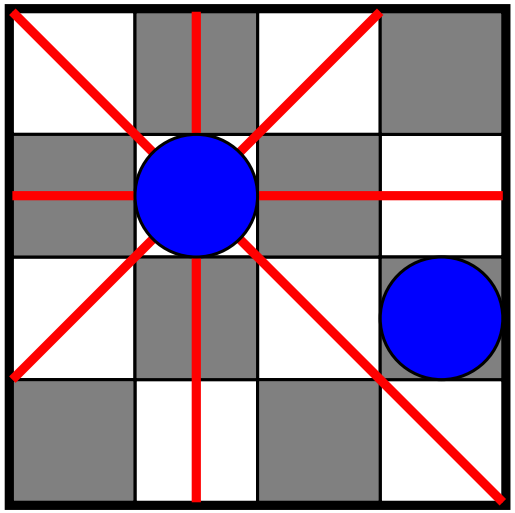


Example

4-queens

4-Queens: put 4 queens on a chessboard 4×4 in such a way that they do not attack one other

Let's start modeling the problem: Variables?



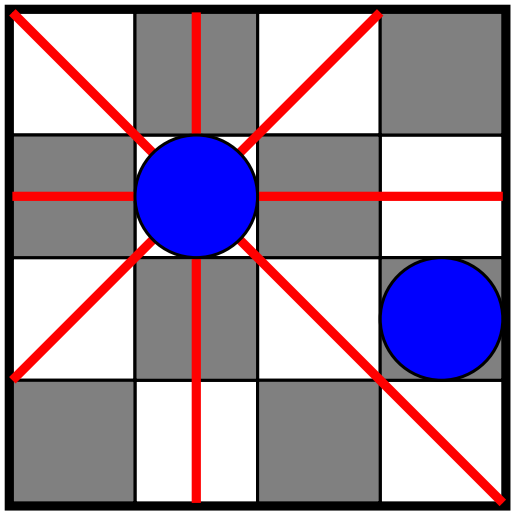
Example

4-queens

4-Queens: put 4 queens on a chessboard 4×4 in such a way that they do not attack one other

Let's start modeling the problem: Variables?

Queens... But how to model them (by 2D coordinates, array of y coordinates)?



Example

4-queens

- Variables

$$X_1, \ldots, X_4$$

$X_i = j$ means "a queen is in column i, row j"

- Domains $$D(X_i) = \{1, \ldots, 4\} i = 1..4$$
- Constraints ($i, j = 1..4$ e $i < j$)

$$X_i \neq X_j \quad \text{(horizontal attack)}$$
$$|X_j - X_i| \neq j - i \quad \text{(diagonal attack)}$$

Try to model the same problem by using $X_{i,j}$ queen model

Example

4-queens

- Variables

$$X_1, \ldots, X_4$$

$X_i = j$ means "a queen is in column i, row j"

- Domains $$D(X_i) = \{1, \ldots, 4\} i = 1..4$$
- Constraints ($i, j = 1..4$ e $i < j$)

$$X_i \neq X_j \quad \text{(horizontal attack)}$$
$$|X_j - X_i| \neq j - i \quad \text{(diagonal attack)}$$

Try to model the same problem by using $X_{i,j}$ queen model

Example

4-queens

- Variables

$$X_1, \ldots, X_4$$

$X_i = j$ means "a queen is in column i, row j"

- Domains $$D(X_i) = \{1, \ldots, 4\} i = 1..4$$
- Constraints ($i, j = 1..4$ e $i < j$)

$$X_i \neq X_j \quad \text{(horizontal attack)}$$
$$|X_j - X_i| \neq j - i \quad \text{(diagonal attack)}$$

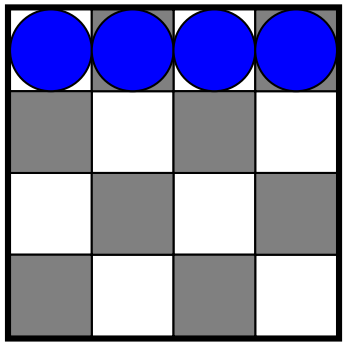
Try to model the same problem by using $X_{i,j}$ queen model

Example - Search for solutions

4-queens

$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 1$$

WRONG



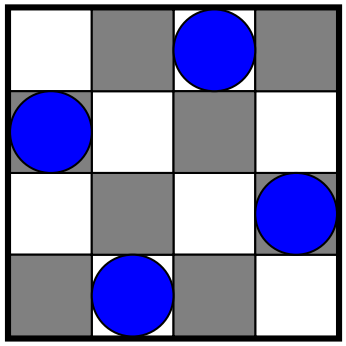
$$\begin{aligned} X_i &\neq X_j && \text{(horizontal)} \\ |X_j - X_i| &\neq j - i && \text{(diagonal)} \end{aligned}$$

Example - Search for solutions

4-queens

$$X_1 = 2, X_2 = 4, X_3 = 1, X_4 = 3$$

CORRECT



$$\begin{aligned} X_i &\neq X_j && \text{(horizontal)} \\ |X_j - X_i| &\neq j - i && \text{(diagonal)} \end{aligned}$$

Constraint Satisfaction Problem

CSP (Constraint Satisfaction Problem)

- Set of Variables $\mathcal{V} = \{V_1, \ldots, V_n\}$
- Set of domains $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_n\}$
- Set of Constraints C on the variables $\mathcal{V}$.
- We look for one (or every) admissible solution, namely an assignment $\sigma : \mathcal{V} \longrightarrow \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_n$ that satisfies all domains and constraints.

No restrictions on domains and on the type of constraints

Declarative programming

Declarative's way (w.r.t. imperative)

- Focus on model (rather than on the algorithm)
- Compositionality of constraints (rather than code rewriting)
- Shorter (but more dense) code

Problem modeling and solving

- Model design
- Implementation (Minizinc, high level)
- Run the Constraint solver
- Model refinement (achieve better performances)
- Incremental implementation / feedback from stakeholders / fast prototyping

The solver

- Behind the scenes
- Algorithms for handling search/optimization
- Constraints interact with solver for guiding the search
- Infeasible solutions are pruned
- Algorithms for efficient propagation of information
- Used as (tunable) black box

Constraint Optimization Problem

COP (Constraint Optimization Problem)

- Set of Variables $\mathcal{V} = \{V_1, \ldots, V_n\}$
- Set of domains $\mathcal{D} = \{\mathcal{D}_1, \ldots, \mathcal{D}_n\}$
- Set of Constraints C on the variables $\mathcal{V}$.
- A function $f : \mathcal{D}^n \longrightarrow \mathbb{A}$
- We look for one (or every) assignment σ that is an admissible solution of the CSP and that minimizes (equivalently, maximizes) the value of f.

No restrictions on domains, kind of constraints, and on the function f

Search space

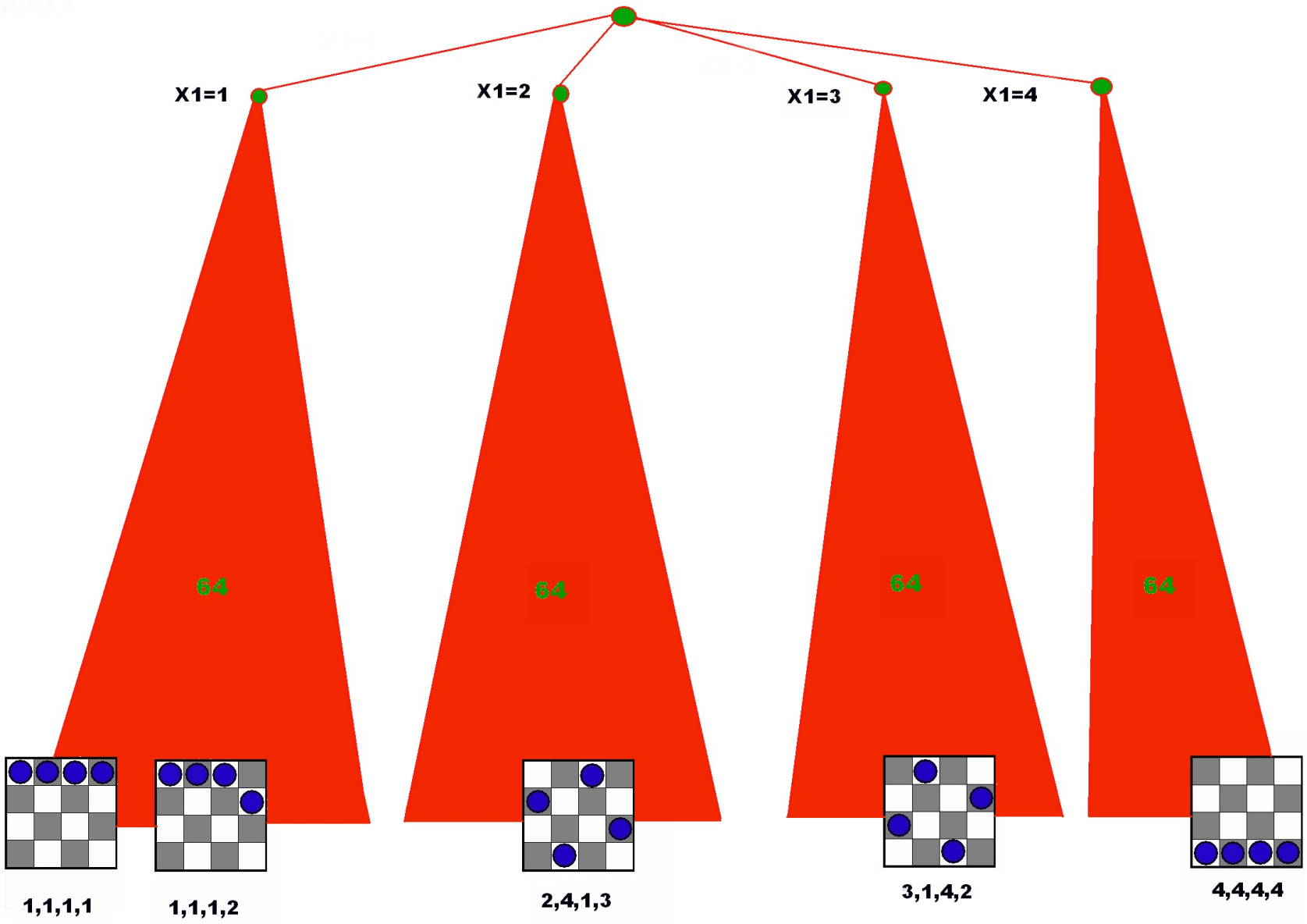
- Given a CSP $\langle X_1 \in \mathcal{D}_1, \ldots, X_n \in \mathcal{D}_n; C \rangle$ (C denotes the set of all the constraints)
- The search space is the set of the n-tuples of domain elements:

$$\mathcal{D}_1 \times \cdots \times \mathcal{D}_n$$

- In this set we look for the points that satisfy the C
- This set is commonly represented as a tree, the search tree.
- Let us observe that, in general, $\mathcal{D}_i$ can be infinite.

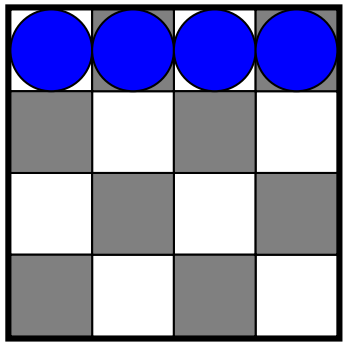
Search space

4-queens



Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

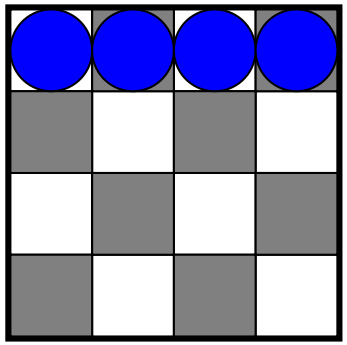
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 1$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

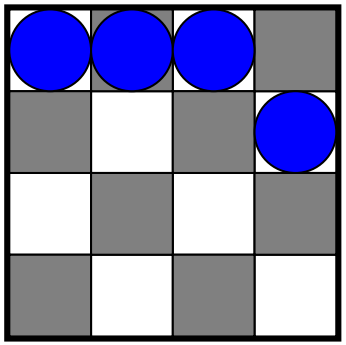
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 1$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

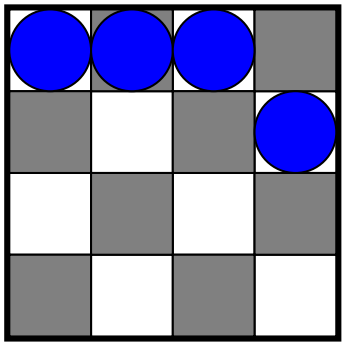
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 2$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

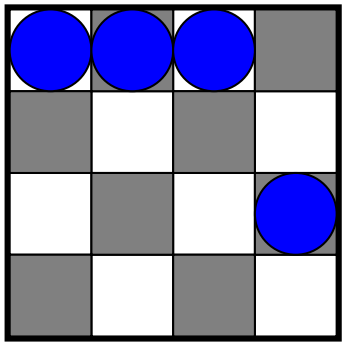
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 2$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

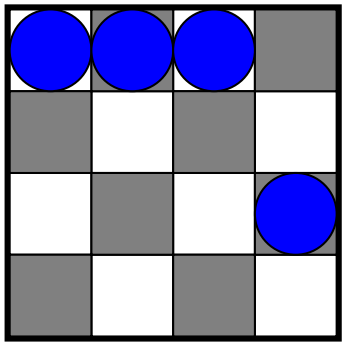
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 3$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

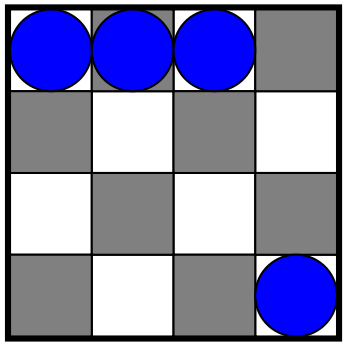
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 3$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

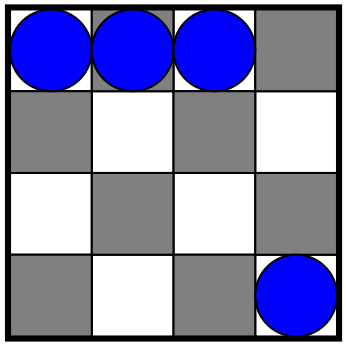
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 4$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

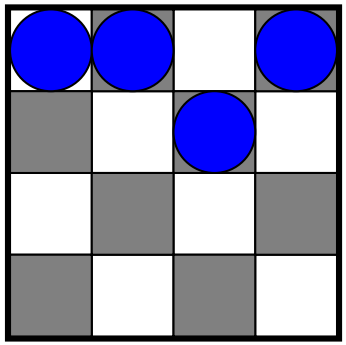
$$X_1 = 1, X_2 = 1, X_3 = 1, X_4 = 4$$



not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

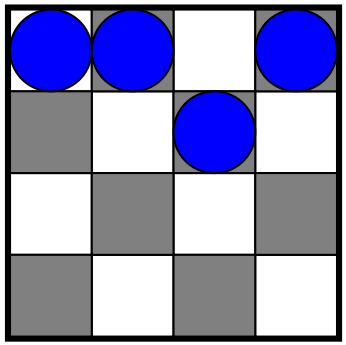
$$X_1 = 1, X_2 = 1, X_3 = 2, X_4 = 1$$



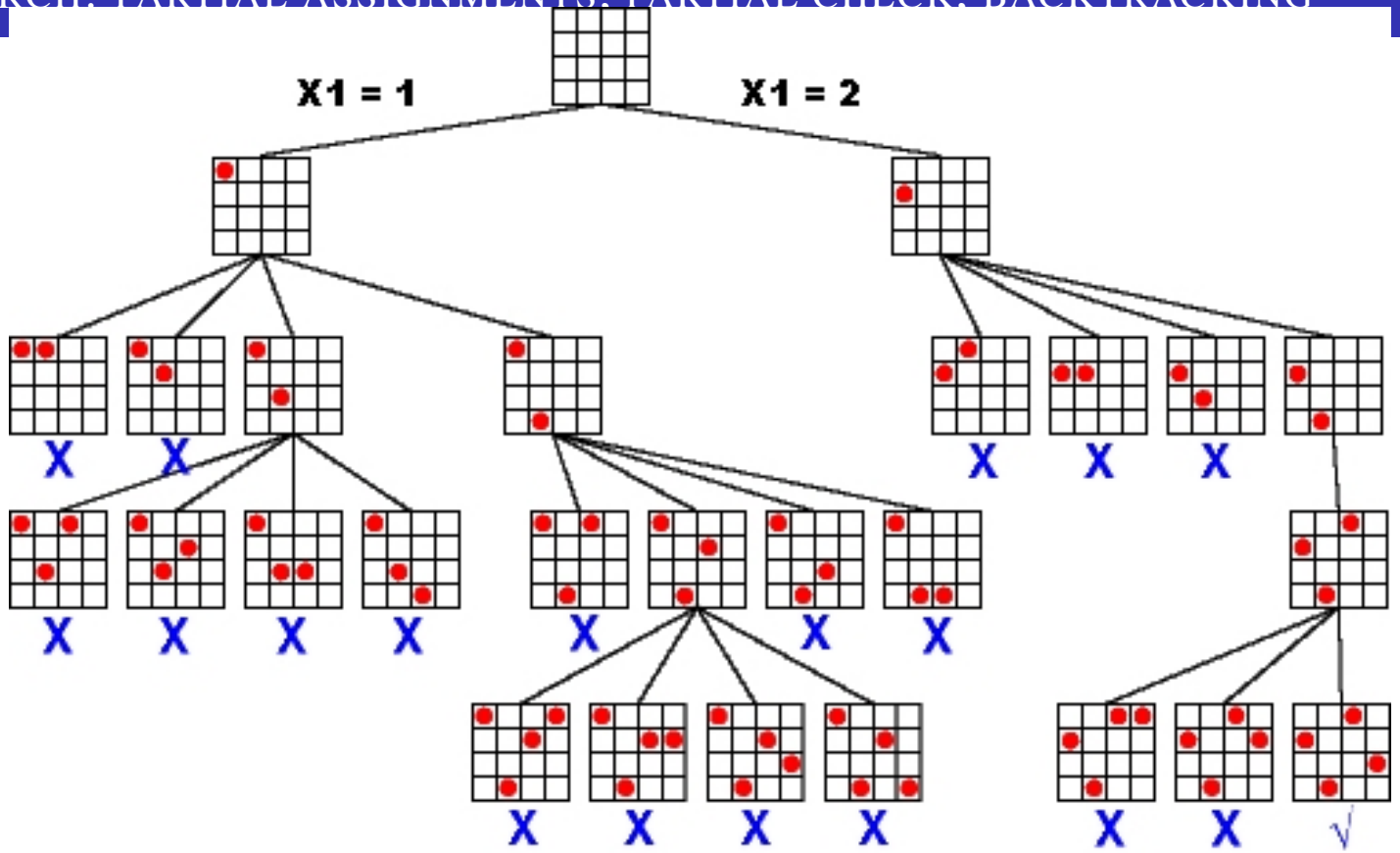
not a solution

Non deterministic technique in which the potential solutions are generated sequentially and verified (akin to Guess & Verify of NP).

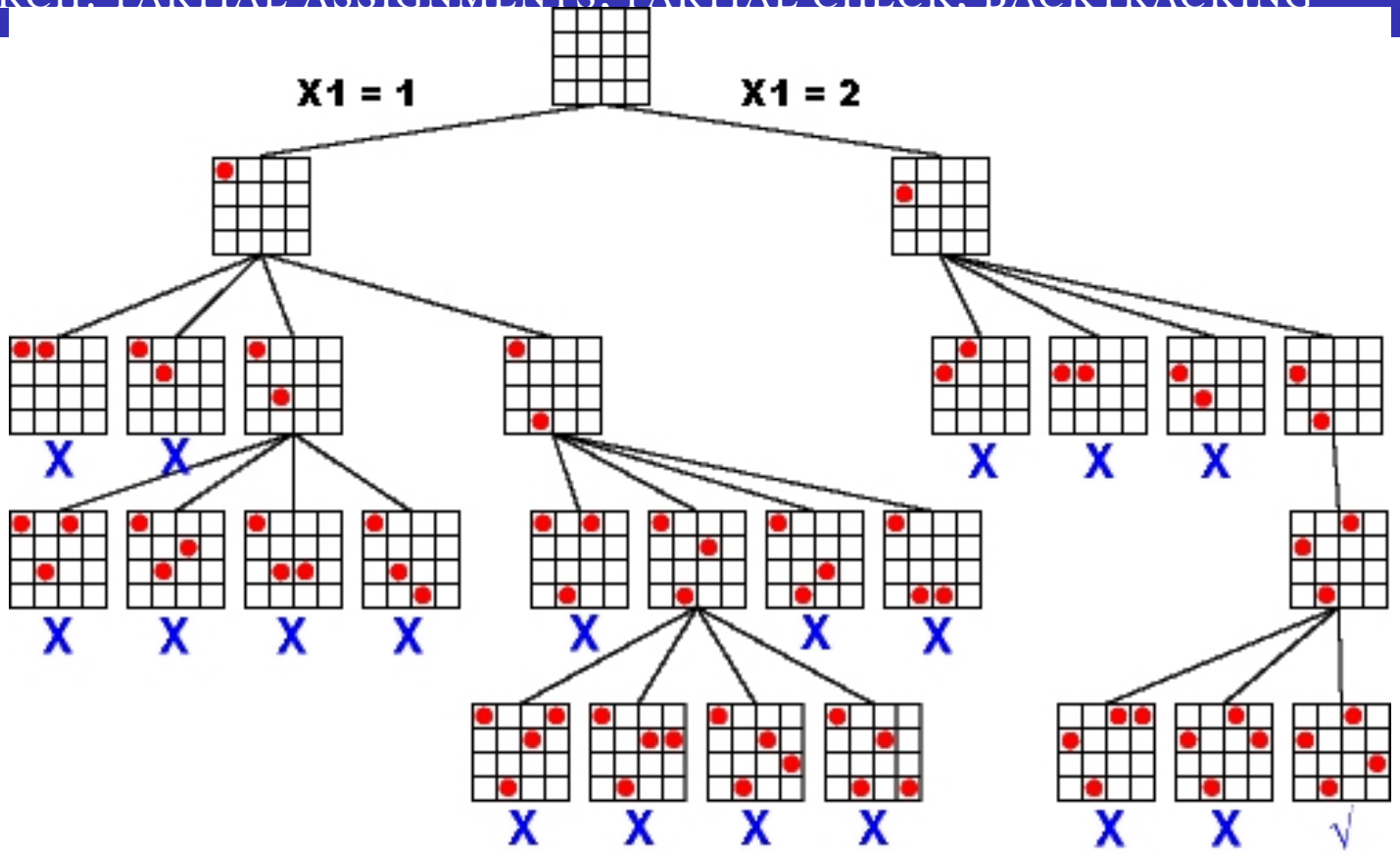
$$X_1 = 1, X_2 = 1, X_3 = 2, X_4 = 1$$



not a solution . . . We are visiting all the tree.
We end up doing $4^4 = 256$ attempts



19 attempts. Assignments with $X_1 = 3$ and $X_1 = 4$ are symmetrical wrt those already visited. Symmetry breaking will be a crucial issue in modeling.



19 attempts. Assignments with $X_1 = 3$ and $X_1 = 4$ are symmetrical wrt those already visited. Symmetry breaking will be a crucial issue in modeling.

CSP/COP

Main "non naive" techniques used for solution's search

- Local Search
- (Integer) Linear Programming
- Translation to SAT and use of a SAT solver
- Constraint Programming
- Answer Set Programming (born for KR purposes, but very effective)

CSP or COP?

COP $\Rightarrow$ CSP

- Assume to have a (black box) COP solver
- Given a CSP $P = \langle X_1 \in \mathcal{D}_1, \ldots, X_n \in \mathcal{D}_n; C \rangle$
- Define f as a constant (e.g. $f(X_1, \ldots, X_n) = 0$)
- Execute the solver of P, f.
- It is clear that the solution of the COP is also a solution of the CSP.

CSP or COP?

CSP $\Rightarrow$ COP

- Suppose now to have a solver (think to it as to a black box) capable of solving a CSP $P' = \langle X_1 \in \mathcal{D}_1, \ldots, X_n \in \mathcal{D}_n; C, f(X_1, \ldots, X_n) \leq k \rangle$
- Let M a (even rough) upper bound of the expected value of the function. Given P' is usually easy to find an upper bound.
- Assume f ranges on $\mathbb{N}$. Using bisection (with a number of calls limited by $\log_2 M$) we can solve the COP through multiple CSPs solving

CSP or COP?

CSP $\Rightarrow$ COP

- Eg, suppose the minimum is 15 (but we don't know it yet) and $M = 205$.
  Let's solve P' with $k = 205, 102, 51, 25, 12$.
  With 12 we have the first no. Thus the minimum is between 13 and 25.
  Let's try $12 + 6$ (yes), $12 + 3$ (yes), $12 + 1$ (no). The doubt is now between 14 or 15.
  Calling the solver on P' with $k = 14$ we discover it.

CSP or COP?

COP $\simeq$ CSP

- In principle solving CSP and COP is not too much different.
- However, finding a solution to large CSPs can be fast (if we are lucky)
- Finding the minimum in large COPs imply the visit of the whole (or, at least of a relevant part of the) search space
- Sometimes we are happy with “good” solutions, not necessarily the minimum.